

Projeto de Sistemas e Gestão de um Sistema Hospitalar

Lucas Saraiva

Tiago Carlos

Introdução

O objetivo deste projeto é desenvolver um protótipo de Sistema de Gestão de Base de Dados para gerenciar algumas tarefas hospitalares, utilizando a linguagem de programação Python e módulos como SQLite3, Pandas e NumPy. O sistema implementa as três principais etapas de ETL (extração, transformação e carregamento) para a integração de dados em um contexto de engenharia de dados. O projeto abrange a gestão de dados de utentes, triagem e consultas médicas, bem como a produção de relatórios estatísticos.

Estrutura da Base de Dados

O sistema foi projetado utilizando uma base de dados relacional SQLite, que é leve e fácil de integrar com aplicações Python. A base de dados contém três tabelas principais:

1. **Tabela Utente:** Armazena informações básicas dos utentes.
 - **HospitalarID:** Chave primária autoincrementada.
 - **ID_SNS:** Identificador único do utente (não chave primária).
 - **Nome, DataNascimento, Genero, Localidade, ContactoTelefonico, DataHoraRegisto:** Informações adicionais do utente.
2. **Tabela Triagem:** Armazena dados coletados durante a triagem.
 - **UtenteID:** Chave estrangeira referenciando **HospitalarID** da tabela **Utente**.
 - **Peso, DoresArticulacoes, AlinhamentoArco, HorasCorridaSemana:** Informações de triagem.
3. **Tabela ConsultaMedica:** Armazena informações de consultas médicas.
 - **UtenteID:** Chave estrangeira referenciando **HospitalarID** da tabela **Utente**.
 - **Idade, SugestaoDiagnostico, DecisaoMedica:** Informações de consultas médicas.

Metodologia

Criação da Base de Dados e Tabelas

A função **create_database** foi implementada para criar a estrutura da base de dados. Utilizando comandos SQL, foram definidas as tabelas com suas respectivas colunas e relações de chave estrangeira.

Inserção de Dados

Para inserir dados na tabela **Utente**, foi criada a função **create_utente**, que recebe um dicionário com as informações do utente e executa um comando SQL para inserir esses dados na tabela.

Atualização de Dados

A função `update_utente` permite a atualização dos dados de um utente existente. Ela recebe o `HospitalarID` do utente e um dicionário com os novos dados, executando um comando SQL de atualização.

Eliminação de Dados

Para eliminar registros de um utente e seus dados associados nas tabelas `Triagem` e `ConsultaMedica`, foi criada a função `delete_utente`. Esta função assegura que todos os dados relacionados ao utente sejam removidos antes de deletar o registro na tabela `Utente`.

Consulta de Dados

A função `get_utente` foi implementada para consultar e retornar as informações de um utente específico com base no `HospitalarID`. Além disso, a função `get_diagnostico` permite consultar a decisão médica sobre o diagnóstico de fascite plantar de um utente.

Inserção de Dados nas Tabelas Triagem e ConsultaMedica

As funções `add_triagem` e `add_consulta_medica` foram criadas para inserir dados nas tabelas `Triagem` e `ConsultaMedica`, respectivamente. Elas recebem os dados necessários e executam comandos SQL para inserir esses dados nas tabelas correspondentes.

Racínio por Trás do Projeto

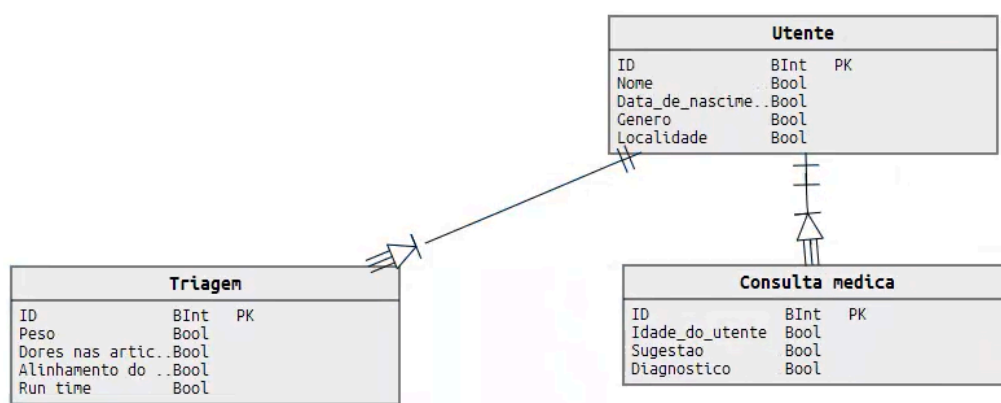
O desenvolvimento deste sistema foi guiado por alguns princípios e requisitos fundamentais:

1. **Modularidade e Reutilização:** Cada função foi projetada para realizar uma tarefa específica, facilitando a manutenção e a reutilização do código.
2. **Integridade Referencial:** O uso de chaves estrangeiras garante a integridade dos dados, assegurando que os registros nas tabelas `Triagem` e `ConsultaMedica` estão sempre associados a um utente válido.
3. **Flexibilidade na Estrutura de Dados:** A escolha de `HospitalarID` como chave primária autoincrementada permite que um utente tenha múltiplas entradas no sistema, facilitando a gestão de múltiplas visitas e consultas.
4. **Simplicidade e Desempenho:** A utilização do SQLite proporciona uma solução simples e eficiente para armazenamento de dados, adequada para o protótipo do sistema hospitalar.

Para facilitar o desenvolvimento das funções de gestão da nossa base de dados, inicialmente criamos seis registros de pacientes (Utentes) para preencher as tabelas correspondentes, excluindo deliberadamente as variáveis de decisão. Essa abordagem foi adotada com o entendimento de que tais variáveis seriam preenchidas posteriormente por meio de um modelo de classificação.

Nosso modelo de classificação se baseia nas variáveis registradas na tabela de triagem, as quais incluem peso, presença de dores nas articulações, alinhamento do pé e o número de horas corridas por semana. Utilizando esses dados como entrada, nosso modelo é capaz de realizar previsões sobre determinadas decisões médicas ou diagnósticos.

Uma vez que o modelo faz suas previsões, os resultados são então incorporados à base de dados. Essa integração de dados é essencial para manter um registro completo e atualizado das informações médicas dos pacientes, permitindo que profissionais de saúde tenham acesso a dados relevantes para auxiliar em seus diagnósticos e tomadas de decisão clínica.



Inicialmente, optamos por não considerar a tabela do secretariado, pois esta é uma entidade fraca, dependendo diretamente da existência da entidade do Utente. Nossa decisão baseou-se no fato de que, sem a existência dos pacientes (Utentes), o papel do secretariado no sistema seria redundante, e assim adicionamos as suas variáveis (ID, data e hora e contacto telefónico) à tabela utente.

Embora pudéssemos aplicar o mesmo raciocínio às tabelas de triagem e consulta médica, optamos por mantê-las no nosso esquema de banco de dados. A razão para essa decisão reside no papel fundamental dessas tabelas na coleta e manutenção de dados clínicos e de saúde dos pacientes. A tabela de triagem, por exemplo, registra informações cruciais sobre a condição inicial do paciente, enquanto a tabela de consulta médica mantém registros dos diagnósticos e tratamentos prescritos.

Quanto à cardinalidade dessas tabelas, decidimos estabelecer que todas as triagens e consultas estão associadas a um único Utente. Essa escolha reflete a prática comum em ambientes hospitalares, onde cada procedimento médico está diretamente ligado a um paciente específico. Ao manter essa cardinalidade simples e direta, simplificamos a estrutura do banco de dados e facilitamos a recuperação e o gerenciamento de informações clínicas dos pacientes.

Portanto, nossa abordagem na concepção do esquema de banco de dados priorizou a simplicidade, a eficiência e a coesão dos dados, garantindo que as tabelas atendessem às necessidades essenciais de registro e gestão de informações médicas, enquanto mantinham uma estrutura lógica e intuitiva.

Conclusão

O protótipo do Sistema de Gestão Hospitalar desenvolvido neste projeto demonstra uma abordagem eficiente e modular para gerenciar dados hospitalares. A utilização de SQLite, juntamente com as funções Python, proporciona uma solução robusta para a gestão de utentes, triagens e consultas médicas, mantendo a integridade e a flexibilidade dos dados. Este sistema pode ser expandido e adaptado para atender a requisitos adicionais e aumentar a complexidade, conforme necessário, em um ambiente hospitalar real.